AI-Powered Product Description Generator

Project Overview

This project uses Generative AI and a Large Language Model (LLM) to create compelling product descriptions for retail and e-commerce applications.

Users enter a product name and keywords, select a writing tone, and receive an AI-generated description.

Project Objectives

- Use a pretrained Large Language Model.
- Generate product descriptions using prompt engineering.
- Build a Python Flask web application.
- Create an interactive HTML interface.
- Personalize descriptions using different writing tones.
- Explore applications of AI in retail and e-commerce.

Technologies

- Python
- Google Colab
- Flask
- Hugging Face Transformers
- PyTorch
- FLAN-T5
- HTML
- CSS

Step 1 – Setup and install libraries

```
!pip install flask transformers torch
```

```
Requirement already satisfied: flask in /usr/local/lib/python3.13/dist-packages (3.1.3)
Requirement already satisfied: transformers in /usr/local/lib/python3.13/dist-packages (5.21.0)
Requirement already satisfied: torch in /usr/local/lib/python3.13/dist-packages (2.11.0+cpu)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.13/dist-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.13/dist-packages (from flask) (8.5.0)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.13/dist-packages (from flask) (2
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.13/dist-packages (from flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.13/dist-packages (from flask) (3.0
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from flask) (3.1.9
Requirement already satisfied: huggingface-hub<3.0,>=1.31.0 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.13/dist-packages (from transformers) (2.
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from transformers)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.13/dist-packages (from transformers) (6.
Requirement already satisfied: regex>=2025.10.22 in /usr/local/lib/python3.13/dist-packages (from transformer
Requirement already satisfied: tokenizers<0.24.0,>=0.23.1 in /usr/local/lib/python3.13/dist-packages (from tr
Requirement already satisfied: typer in /usr/local/lib/python3.13/dist-packages (from transformers) (0.27.3)
Requirement already satisfied: safetensors>=0.8.0 in /usr/local/lib/python3.13/dist-packages (from transforme
Requirement already satisfied: tqdm>=4.60 in /usr/local/lib/python3.13/dist-packages (from transformers) (4.6
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-
Requirement already satisfied: hf-xet<2.0.0,>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from huggingf
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.13/dist-packages (from hugg
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.0->huggi
Requirement already satisfied: certifi in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.0->hug
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.
Requirement already satisfied: idna in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.0->huggin
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->http
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.13/dist-packages (from torch) (80.10.2
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-packages (from torch) (1.14.0)
```

```
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.13/dist-packages (from torch) (3.7)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from sympy>=1.1
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from typer->tra
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.13/dist-packages (from typer->transform
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.13/dist-packages (from typer->t
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.13/dist-packages (from rich>=1
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages (from rich>
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.13/dist-packages (from markdown-it-py>=2.
```

```
from flask import Flask, request, render_template
from transformers import pipeline
import torch
import os
import subprocess
import time
```

Step 2 – LLM integration

```
generator = pipeline(
    "text-generation",
    model="gpt2"
)
```

```
Loading weights: 100%                                                    148/148 [00:00<00:00, 2154.32it/s]
```

Step 3 – Prompt Engineering & Description Generation Testing

```
def generate_description_transformers(product_name, keywords, tone="Professional"):
    keywords_str = ", ".join(keywords) if isinstance(keywords, list) else keywords
    prompt = f"Write a {tone} product description for {product_name} with the following keywords: {keywords_

    outputs = generator(
        prompt,
        max_new_tokens=60,
        num_return_sequences=1,
        do_sample=True,
        temperature=0.7
    )
    return outputs[0]["generated_text"][len(prompt):].strip()

# Test the function
print(generate_description_transformers(
    "Leather Boots",
    ["durable", "stylish", "waterproof"],
    "formal"
))
```

```
Leather Boot is manufactured by a company in Romania for the sole of the shoes. When worn with a leather sole
```

Step 3 – Create your Flask application

```
%%writefile app.py

from flask import Flask, request, render_template
from transformers import pipeline
import torch

app = Flask(__name__)

# Load the AI model only once
generator = pipeline(
    "text-generation",
    model="gpt2"
)
```

```
def generate_description_transformers(product_name, keywords, tone="neutral"):

    prompt = f"""
    Write a compelling product description for {product_name}.
    Include these keywords: {keywords}.
    Use a {tone} tone.
    Product description:
    """

    generated = generator(
        prompt,
        max_new_tokens=100,
        num_return_sequences=1,
        pad_token_id=generator.tokenizer.eos_token_id,
        do_sample=True,
        temperature=0.7
    )

    description = generated[0]["generated_text"]

    return description[len(prompt):].strip()


@app.route("/", methods=["GET", "POST"])
def index():

    description = None
    error = None

    if request.method == "POST":

        product_name = request.form.get("product_name", "").strip()
        keywords = request.form.get("keywords", "").strip()
        tone = request.form.get("tone", "neutral")

        if not product_name or not keywords:
            error = "Please enter a product name and keywords."

        else:
            try:
                description = generate_description_transformers(
                    product_name,
                    keywords,
                    tone
                )
            except Exception as e:
                error = "Unable to generate description. Please try again."
                print("Error:", e)

    return render_template(
        "index.html",
        description=description,
        error=error
    )


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=False)
```

```
Overwriting app.py
```

Step 4 – Create your HTML template

```
import os

os.makedirs("templates", exist_ok=True)

html_code = """
<!DOCTYPE html>
<html>
```

```
<head>
    <title>AI Product Description Generator</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f2f4f8;
            margin: 40px;
        }

        .container {
            background: white;
            max-width: 650px;
            margin: auto;
            padding: 30px;
            border-radius: 12px;
        }

        h1 {
            text-align: center;
            color: #263b5e;
        }

        label {
            font-weight: bold;
        }

        input, select {
            width: 100%;
            padding: 12px;
            margin: 10px 0 20px;
            box-sizing: border-box;
        }

        button {
            width: 100%;
            padding: 12px;
            background: #263b5e;
            color: white;
            border: none;
            cursor: pointer;
        }

        .result {
            background: #eaf2ff;
            padding: 15px;
            margin-top: 20px;
            border-radius: 8px;
        }

        .error {
            color: red;
        }
    </style>
</head>

<body>

<div class="container">

    <h1>AI Product Description Generator</h1>

    <form method="POST">

        <label for="product_name">Product Name:</label>
        <input type="text"
               id="product_name"
               name="product_name"
               placeholder="Example: Leather Boots"
               required>

        <label for="keywords">Keywords (comma-separated):</label>
```

```
            <input type="text"
                   id="keywords"
                   name="keywords"
                   placeholder="Durable, stylish, waterproof"
                   required>

            <label for="tone">Select Writing Tone:</label>

            <select id="tone" name="tone">
                <option value="neutral">Neutral</option>
                <option value="formal">Formal</option>
                <option value="casual">Casual</option>
                <option value="humorous">Humorous</option>
                <option value="for young adults">For Young Adults</option>
                <option value="for professionals">For Professionals</option>
            </select>

            <button type="submit">
                Generate Description
            </button>

        </form>

        {% if description %}
            <div class="result">
                <h2>Generated Description:</h2>
                <p>{{ description }}</p>
            </div>
        {% endif %}

        {% if error %}
            <p class="error">{{ error }}</p>
        {% endif %}

    </div>

    </body>
    </html>
    """

    with open("templates/index.html", "w") as file:
        file.write(html_code)

    print("HTML template created successfully!")
```

```
HTML template created successfully!
```

Step 5 – Run your Flask application

```
import subprocess
import time
from google.colab import output

# Stop an earlier Flask process if one is running
if "process" in globals() and process.poll() is None:
    process.terminate()
    process.wait()

# Start Flask in the background
process = subprocess.Popen(
    ["python", "app.py"],
    stdout=subprocess.PIPE,
    stderr=subprocess.STDOUT,
    text=True
)

# Give Flask time to start
time.sleep(15)

# Check whether Flask started
if process.poll() is not None:
```

```
    print(process.stdout.read())
else:
    print("Flask application is running!")

    # Display the Colab website link
    print(output.eval_js("google.colab.kernel.proxyPort(5000)"))
```

```
Flask application is running!
https://5000-m-s-usw4b1-3bpyuqn8lcc4o-b.us-west4.prod.r.colab.dev
```